

View Binding no Android

Professor: Henrique Galati
IFSP - Campus Araraquara
Análise e Desenvolvimento de Sistemas

View Binding é uma funcionalidade do Android que **gera uma classe automática** para cada layout XML do seu app. Essa classe fornece **acesso direto e seguro às views**, evitando a necessidade de usar `findViewById()`. Com View Binding, seu código fica mais simples, mais seguro e mais organizado.

Por que evitar o `findViewById()`?

O método `findViewById(R.id.algo)` era a forma padrão de acessar elementos da interface gráfica. Embora funcione, ele possui várias **desvantagens importantes**:

Verbosidade: Você precisa escrever muito mais código repetitivo e repetir isso para cada botão, texto, imagem. Exemplo do aplicativo Perfil:

```
val btnDayNight = findViewById<Button>(R.id.btnDayNight)
val btnIdioma = findViewById<Button>(R.id.btnIdioma)
val btnAlterar = findViewById<Button>(R.id.btnAlterar)
val etNome = findViewById<EditText>(R.id.etNome)
val etEmail = findViewById<EditText>(R.id.etEmail)
val tvNome = findViewById<TextView>(R.id.tvNome)
val tvEmail = findViewById<TextView>(R.id.tvEmail)
```

Erro de tipo (cast errado): Se você usar o tipo errado (ex.: `EditText` no lugar de `TextView`), o projeto pode compilar, mas **quebrar em tempo de execução** com `ClassCastException`.

Risco de NullPointerException: Se você chamar `findViewById` antes do `setContentview()`, ou se o ID não existir no layout, a View será `null` — e seu app pode travar com `NullPointerException`.

Refatoração insegura: Se você **renomear um ID no XML**, o `findViewById(R.id.algo)` **continua existindo**, mas aponta para algo incorreto ou

inexistente. Isso pode causar bugs difíceis de detectar. Projetos com dezenas de `findViewById` espalhados se tornam difíceis de manter e refatorar.

Como ativar o View Binding no Android Studio

No `build.gradle` (Module: app), adicione o código abaixo e depois clique em “**Sync Now**”.

```
android {  
    ...  
    buildFeatures {  
        viewBinding = true  
    }  
}
```

Suponha que você tenha o layout `activity_main.xml` com botão `btnSalvar`. Com o View Binding ligado, o Android Studio gerará automaticamente a classe `ActivityMainBinding`. Perceba que o nome segue a convenção do que está no XML, retirando o `_` e adicionando Binding no final. Para usarmos o binding na classe MainActivity, você deverá declarar o binding e inflar o layout nele:

```
class MainActivity : AppCompatActivity() {  
    private lateinit var binding: ActivityMainBinding  
  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
  
        binding = ActivityMainBinding.inflate(layoutInflater)  
        setContentView(binding.root)  
  
        binding.btnSalvar.setOnClickListener {  
            // Código do listener  
        }  
    }  
}
```

Vantagens do uso do View Binding

- A classe de binding já sabe **quais Views existem** e seus tipos corretos. Não há necessidade de cast manual. Se você errar o nome, o app **nem compila**.
- Você escreve menos código, pois não há a necessidade de criar variáveis extras para cada componente. **Diminui a necessidade de escrever código boilerplate**.
- O View Binding é fácil de ativar e não exige bibliotecas externas.

- Aumenta a manutenibilidade do código, pois o código fica mais desacoplado.

Refatorando novamente o Perfil App

Na última aula, refatoramos a classe `MainActivity` do aplicativo Perfil, aplicando as **convenções de código** para `Activities`. Vamos refatorar esta classe mais uma vez, **aplicando o View Binding**. Abaixo, o código atual da classe. Em seguida, o código refatorado com o uso do View Binding. Não esqueça de ativá-lo no build.gradle.

1ª versão da MainActivity

```
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val btnDayNight = findViewById<Button>(R.id.btnDayNight)
        val btnIdioma = findViewById<Button>(R.id.btnIdioma)
        val btnAlterar = findViewById<Button>(R.id.btnAlterar)
        val etNome = findViewById<EditText>(R.id.etNome)
        val etEmail = findViewById<EditText>(R.id.etEmail)
        val tvNome = findViewById<TextView>(R.id.tvNome)
        val tvEmail = findViewById<TextView>(R.id.tvEmail)

        btnDayNight.setOnClickListener {
            val isNight = AppCompatDelegate.getDefaultNightMode() ==
                AppCompatDelegate.MODE_NIGHT_YES

            val mode = if (isNight) {
                AppCompatDelegate.MODE_NIGHT_NO
            } else {
                AppCompatDelegate.MODE_NIGHT_YES
            }

            AppCompatDelegate.setDefaultNightMode(mode)
        }

        btnIdioma.setOnClickListener {
            val idiomaAtual = AppCompatDelegate.getApplicationLocales()
            val usandoIdiomaPadrao =
                !idiomaAtual.isEmpty && idiomaAtual[0]?.language == "en"
            val proximoIdioma = if (usandoIdiomaPadrao) "pt" else "en"
            AppCompatDelegate.setApplicationLocales(
                LocaleListCompat.forLanguageTags(proximoIdioma)
            )
        }

        btnAlterar.setOnClickListener {
            tvNome.text = etNome.text.toString()
        }
    }
}
```

```

        tvEmail.text = etEmail.text.toString()
    }
}
}

```

2ª versão da MainActivity

```

class MainActivity : AppCompatActivity() {

    companion object {
        const val TAG = "MainActivity"
    }

    private lateinit var btnDayNight: Button
    private lateinit var btnIdioma: Button
    private lateinit var btnAlterar: Button
    private lateinit var etNome: EditText
    private lateinit var etEmail: EditText
    private lateinit var tvNome: TextView
    private lateinit var tvEmail: TextView

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        setupViews()
        setupListeners()
    }

    private fun setupViews() {
        btnDayNight = findViewById(R.id.btnDayNight)
        btnIdioma = findViewById(R.id.btnIdioma)
        btnAlterar = findViewById(R.id.btnAlterar)
        etNome = findViewById(R.id.etNome)
        etEmail = findViewById(R.id.etEmail)
        tvNome = findViewById(R.id.tvNome)
        tvEmail = findViewById(R.id.tvEmail)
    }

    private fun setupListeners() {
        btnDayNight.setOnClickListener { mudarTema() }
        btnIdioma.setOnClickListener { alternarIdioma() }
        btnAlterar.setOnClickListener { atualizarTexto() }
    }

    private fun mudarTema() {
        val isNight = AppCompatDelegate.getDefaultNightMode() ==
            AppCompatDelegate.MODE_NIGHT_YES

        val mode = if (isNight) {
            AppCompatDelegate.MODE_NIGHT_NO
        } else {
            AppCompatDelegate.MODE_NIGHT_YES
        }

        AppCompatDelegate.setDefaultNightMode(mode)
    }
}

```

```

    }

    private fun alternarIdioma() {
        val idiomaAtual = AppCompatDelegate.getApplicationLocales()
        val usandoIngles = !idiomaAtual.isEmpty && idiomaAtual[0]?.language == "en"
        val proximoIdioma = if (usandoIngles) "pt" else "en"

        AppCompatDelegate.setApplicationLocales(
            LocaleListCompat.forLanguageTags(proximoIdioma))
    }

    private fun atualizarTexto() {
        tvNome.text = etNome.text.toString()
        tvEmail.text = etEmail.text.toString()
    }
}

```

Versão final da MainActivity

```

class MainActivity : AppCompatActivity() {

    companion object {
        const val TAG = "MainActivity"
    }

    private lateinit var binding: ActivityMainBinding

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        binding = ActivityMainBinding.inflate(layoutInflater)
        setContentView(binding.root)
        setupListeners()
    }

    private fun setupListeners() {
        binding.btnDayNight.setOnClickListener { mudarTema() }
        binding.btnIdioma.setOnClickListener { alternarIdioma() }
        binding.btnAlterar.setOnClickListener { atualizarTexto() }
    }

    private fun mudarTema() {
        val isNight = AppCompatDelegate.getDefaultNightMode() ==
            AppCompatDelegate.MODE_NIGHT_YES

        val novoModo = if (isNight) {
            AppCompatDelegate.MODE_NIGHT_NO
        } else {
            AppCompatDelegate.MODE_NIGHT_YES
        }

        AppCompatDelegate.setDefaultNightMode(novoModo)
    }

    private fun alternarIdioma() {

```

```
val idiomaAtual = AppCompatActivity.getApplicationLocales()
val usandoIngles = !idiomaAtual.isEmpty && idiomaAtual[0]?.language == "en"
val proximoIdioma = if (usandoIngles) "pt" else "en"

AppCompatActivity.setApplicationLocales(
    LocaleListCompat.forLanguageTags(proximoIdioma)
)
}

private fun atualizarTexto() {
    val nome = binding.etNome.text.toString()
    val email = binding.etEmail.text.toString()

    binding.tvNome.text = nome
    binding.tvEmail.text = email
}
}
```

Exercício Prático: Refatorar o conversor de dólar para real

Na última aula, desenvolvemos um aplicativo para a conversão de dólar para real e vice-versa. **Refatore o aplicativo usando o View Binding.**